Model Checking
&
Logiques Temporelles

LOS

Chapitre 5

Détails du Chapitre 5

- Définitions générales
  - Systèmes réactifs
  - Propriétés temporelles
  - Systèmes de transitions
  - Automates
- Langage de la logique temporelle
- Syntaxe de la logique temporelle
- Sémantique de la logique temporelle

Définitions générales

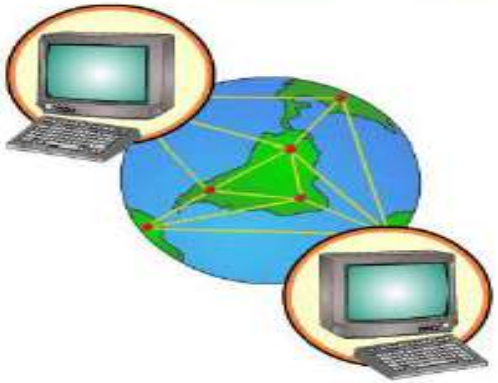
Définitions générales

Systèmes réactifs

- Programme Standard
  - Se termine, retourne un résultat, manipule des données complexes
  - Sa structure de contrôle est assez simple (propriétés à prouver à l'aide de la LH)
  - Exemples : compilateur, algorithme de tri, etc.
- Systèmes réactifs
  - Ils ne se terminent pas forcément
  - Ils ne calculent pas un résultat (mais plutôt maintiennent une interaction)
  - Les types de données manipulés sont souvent simples, mais le contrôle est complexe (exécution de plusieurs composants en parallèle)
  - Ils interagissent avec un environnement par le biais de capteurs (prise d'info) et d'actionneurs (action)
  - Exemple : Systèmes embraqués pour transport/énergie, OS, Protocoles de communication, Système ascenseur, feux de croisement, ...

Définitions générales

→ Systèmes réactifs



Définitions générales

→ Propriétés temporelles

- Programme Standard
  - Propriétés à vérifier impliquent des prédicats très riches sur les données manipulées (ex. tableau est trié)
  - Aspects temporels très restreints (toujours identiques)
- Systèmes réactifs
  - Aspect temporel est très varié (relations temporelles de précédences, …)
  - Prédicats sur les données sont souvent simples (ex. $x \neq 0$)
    - ✓ Si un processus demande infiniment souvent à être exécuté alors l'OS finira par l'exécuter
    - ✓ Chaque fois qu'une panne est détectée, une alarme est émise

➢ Pour exprimer des propriétés temporelles, on utilise des logiques temporelles

- Décrivent les comportements temporels d'une manière non ambiguë
- Souplesse et généricité des algorithmes exprimables dans une certaine logique

Définitions générales

Model-Checking

- Une technique de la vérification formelle
  - Pour prouver formellement la correction d'une spécification ou d'un programme
  - Utilisée pour vérifier des logiciels en particulier le logiciel temps réel et critique
- Ingrédients :
  - Une formule logique F (en logique LTL, CTL, etc.)
  - Un modèle/abstraction finie du système, sous forme d'un système de transition S
  - Question : Est-ce que S satisfait F? ou est ce que S est un modèle pour F

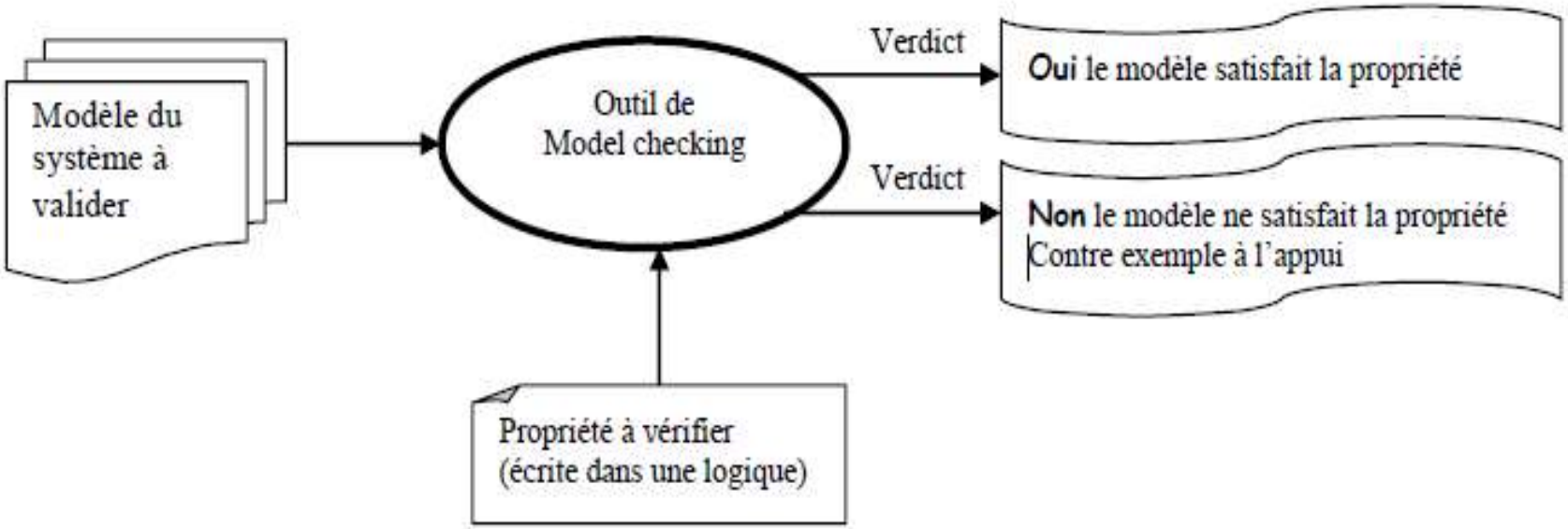
$$S \models F?$$

- D'où le nom de Model-Checking

- Les outils de model-checking rendent un verdict : *oui* (la propriété est satisfaite) ou non. En général, les outils fonctionnent à l'aide de contre exemples.

Définitions générales

→ Model-Checking



Principe simplifié du model-checking

Définitions générales

→ Propriétés temporelles

- Programme Standard
  - Propriétés à vérifier impliquent des prédicats très riches sur les données manipulées (ex. tableau est trié)
  - Aspects temporels très restreints (toujours identiques)
- Systèmes réactifs
  - Aspect temporel est très varié (relations temporelles de précédences, …)
  - Prédicats sur les données sont souvent simples (ex. $x \neq 0$)
    - ✓ Si un processus demande infiniment souvent à être exécuté alors l'OS finira par l'exécuter
    - ✓ Chaque fois qu'une panne est détectée, une alarme est émise

➢ Pour exprimer des propriétés temporelles, on utilise des logiques temporelles

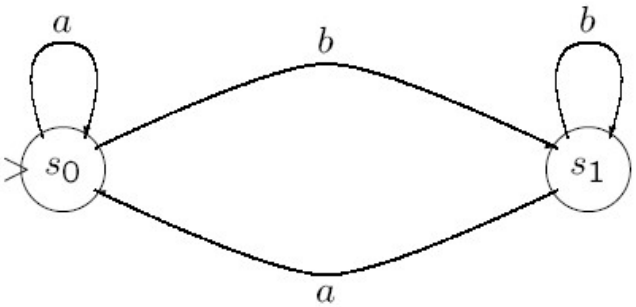
- Décrivent les comportements temporels d'une manière non ambiguë
- Souplesse et généricité des algorithmes exprimables dans une certaine logique

Définitions générales

Systèmes de transitions (ST)

- ST permet de représenter le comportement d'un ensemble de processus (d'un système concurrent, d'un système réactif)
- Il est défini par un quadruplet **$S = (Q, E, T, q_0)$:** ST de transitions étiquetés
  - Q : Un ensemble fini d'états
  - E : Ensemble fini des étiquettes associées aux transitions
  - $T \subseteq Q \times E \times Q$: Ensemble de transitions
    - ✓ Une transition est une relation entre deux états de Q, étiquetée par un élément de E.
  - q_0 : est l'élément initial, $q_0 \in Q$
- Une transition t est un triplet $t = (p, e, q)$ que l'on note $p \rightarrow^{e} q$

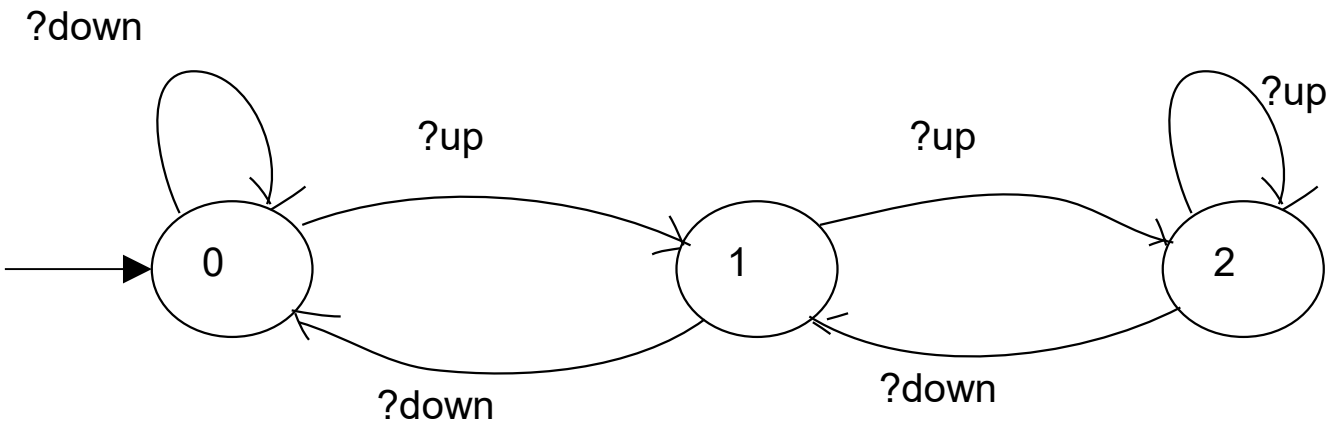
$Q = \{s_0, s_1\}$
$E = \{a, b\}$
$q_0 = s_0$



Définitions générales

Systèmes de transitions (ST) : Exemples

- Système d'un ascenseur :
  - Ensemble de processus : cabine, contrôleur, porte_i
  - 3 étages
  - Système (porte_0 || porte_1 || porte_2 || cabine || contrôleur)
  - cabine :



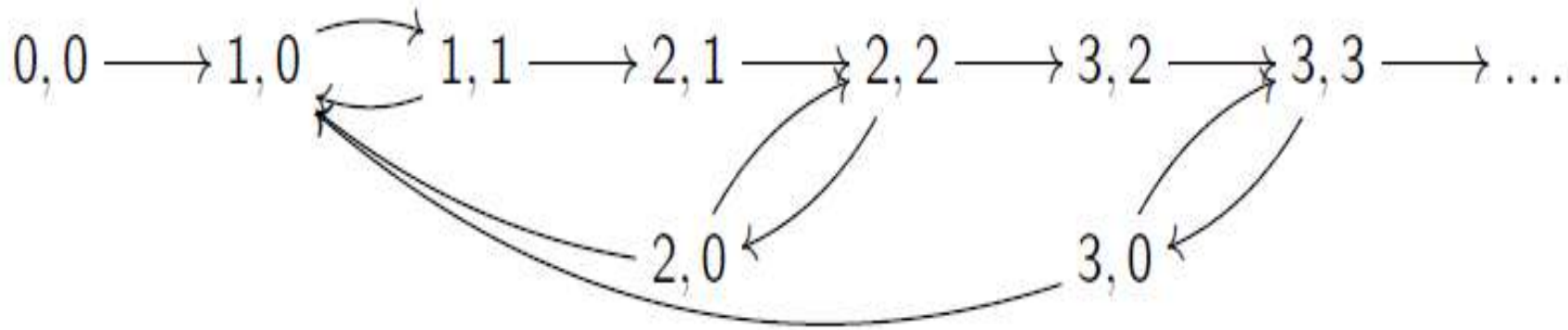
Définitions générales

Systèmes de transitions (ST) : Exemples

- Un système à 3 processus exécutant concurremment (par entrelacement) :

boucle **$x \leftarrow y+1$**	‖	**boucle** **$y \leftarrow x$**	‖	**boucle** **$y \leftarrow 0$**

  - Le système peut-être décrit en termes d'états :

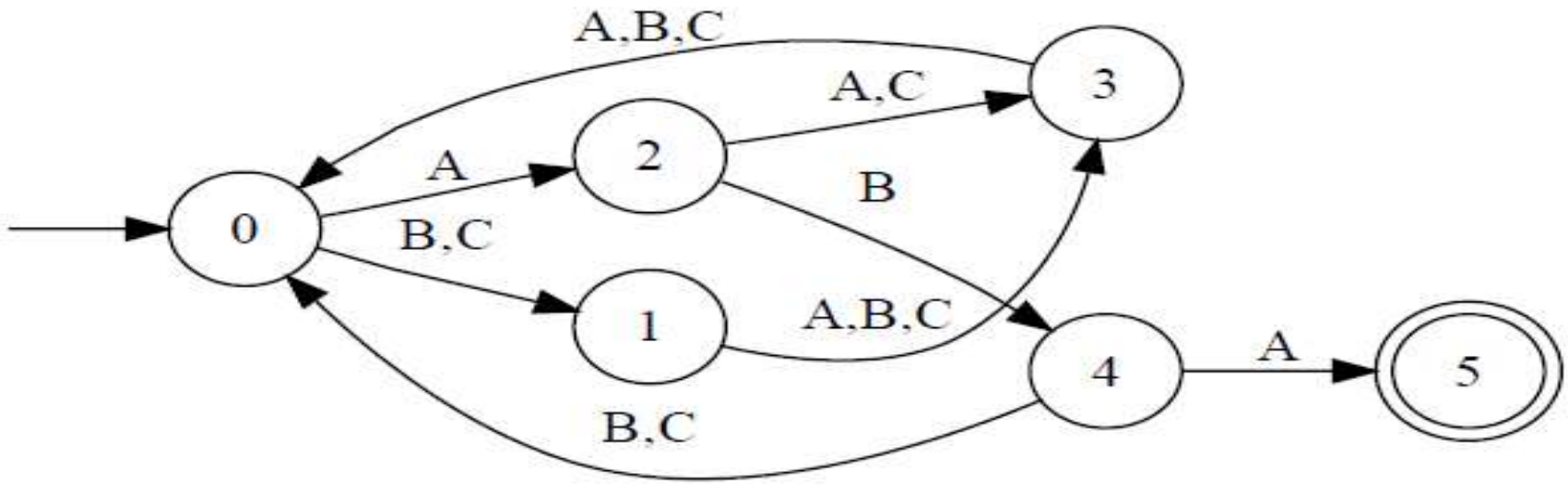


- Propriétés désirables :
  - L'état 4,2 est-il accessible?
  - Le système s'arrête-il? toujours, parfois?
  - Est-il toujours vrai que $y = 0 \vee 0 \leq x - y \leq 1$?

Définitions générales

Systèmes de transitions (ST) : Exemples

- Soit un système de digicode à 3 touches A, B et C
  - La porte s'ouvre quand ABA est saisi
  - Le digicode est dans l'état initial après saisie d'un mauvais code.
- Le digicode peut-être décrit par le ST suivant :



Définitions générales

→ Systèmes de transitions (ST) :

- ST est aussi appelé automate
- Systèmes réactifs présentent souvent des comportements infinis (pas d'états terminaux, nombre d'états possibles infinis, exécution infinie possible)

Système de transitions $\neq$ automate

- Pas d'étiquette sur les transitions (ou comme si)
- Une transition n'est pas causée par l'environnement
- Pas d'états terminaux
- Nombre d'états infini possible
- Exécution infinie possible

Définitions générales

Automates et propriétés : Définitions

- $P = \{P_1, P_2, \dots\}$: ensemble de propositions élémentaires décrivant des propriétés associées aux états
- Un automate, dans ce cas, est un quadruplet **A = (Q, E, T, q_0, l)** où :
  - Q, E, T, q_0 : même définition que précédemment
  - l : application qui associe à tout état de Q l'ensemble fini des propriétés élémentaires vérifiées dans cet état.

$$\boldsymbol{l(q_i) = \{P_k \mid P_k \in P \wedge q_i \vDash P_k\}}$$, $\vDash$: relation de satisfaction

- L'application l permet entre autre de :
  - Décrire les variables d'états qui caractérisent un système de processus
  - Les observer lors d'un changement d'état

Définitions générales

→ Automates et propriétés : Comportement

- Illustré en termes de chemins dans un automate **A = (Q, E, T, q_0, I)**
- Un chemin dans A est une suite σ, finie ou infinie de :
  - transitions (q_i, e_i, q'_i) de A, qui
  - s'enchaînent, c-à-d, $q'_i = q_{i+1}$
- Un chemin est noté $\sigma = q_1 \rightarrow^{e_1} q_2 \rightarrow^{e_2} q_3 \rightarrow^{e_3} q_4 \cdots$
- Longueur de chemin : $|\sigma|$
  - nombre de transitions qu'il contient
  - $|\sigma| \in \mathbb{N} \cup \{\propto\}$: peut-être finie ou non

Définitions générales

Automates et propriétés : Comportement

- En terminologie des systèmes de processus, un chemin est également désigné par le mot trace
- Exécution partielle
  - Chemin partant de l'état initial
- Exécution complète
  - Exécution partielle maximale (qui ne peut être prolongée)

Langage de la logique temporelle

Règle de composition concurrente

- Besoin d'un nouveau modèle
  - Soit un système concurrent formé par deux processus $Proc_1$ et $Proc_2$
  - On est tenté par écrire pour ce programme la règle de composition concurrente suivante (issue d'une extension de la LH)

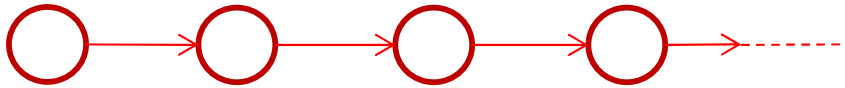
$$\frac{\{p_1\}\ Proc_1\ \{q_1\} \qquad \{p_2\}\ Proc_2\ \{q_2\}}{\{p_1 \wedge p_2\}\ \ Proc_1\ \parallel\ Proc_2\ \ \{q \wedge q_2\}}$$

- Mais, la règle est fausse s'il y a de l'interaction (via des mémoires partagées) entre les processus $Proc_1$ et $Proc_2$
- Besoin donc d'un autre modèle pour raisonner sur les propriétés globales des programmes concurrents (les exprimer, vérifier et prouver)

Langage de la logique temporelle

Logique temporelle (LT)

- LT est un langage décrivant les propriétés des séquencements d'états



  - Chaque état dans la séquence donne une valeur de vérité à des propositions atomiques

- LT est une extension de la logique des propositions

- LT utilise des opérateurs temporels pour parler du futur et du passé

  - Exemples :
    - ✓ p est vrai à l'instant (ou état) suivant
    - ✓ p finit par être vrai dans le futur
    - ✓ Quand je vois p, je vois ensuite q exactement 3 observations après

Langage de la logique temporelle

Logique temporelle :

- Propriétés temporelles : utiles en vérification
  - Accessibilité : Une certaine situation peut être atteinte

    Le compteur x peut valoir 0
  - Sureté : Quelque chose de mauvais n'arrive jamais

    J'accède au fichier uniquement si j'ai entré le bon PIN
  - Invariance : Chaque état local respecte une bonne propriété

    Pas de division par 0, le tableau ne déborde jamais
  - Vivacité : Quelque chose de bon finit par arriver

    Le programme termine, le message finit toujours par être transmis
  - Equité : Quelque chose de bon se répète infiniment souvent (variante de la vivacité)

    si un processus demande toujours la main, il l'aura infiniment

- La plupart des propriétés intéressantes sur les systèmes de processus peuvent s'exprimer comme la conjonction de sûreté et de vivacité.

Langage de la logique temporelle

Logique temporelle : 3 Composantes essentielles

- Propositions :
  - Permettent de caractériser les états qui apparaissent dans les comportements (les séquencements)

    P_i est vraie dans l'état q si $P_i \in I(q)$

- Connecteurs logiques :
  - Connecteurs classiques $\wedge$, $\vee$, $\neg$, $\Rightarrow$ et $\Leftrightarrow$: indispensables pour construire des formules logiques complexes.

- Connecteurs temporels :
  - Permettent de décrire les séquencements des états/événements observés le long d'une exécution.

Langage de la logique temporelle

Logique temporelle : 3 Composantes essentielles

- Connecteurs temporels :

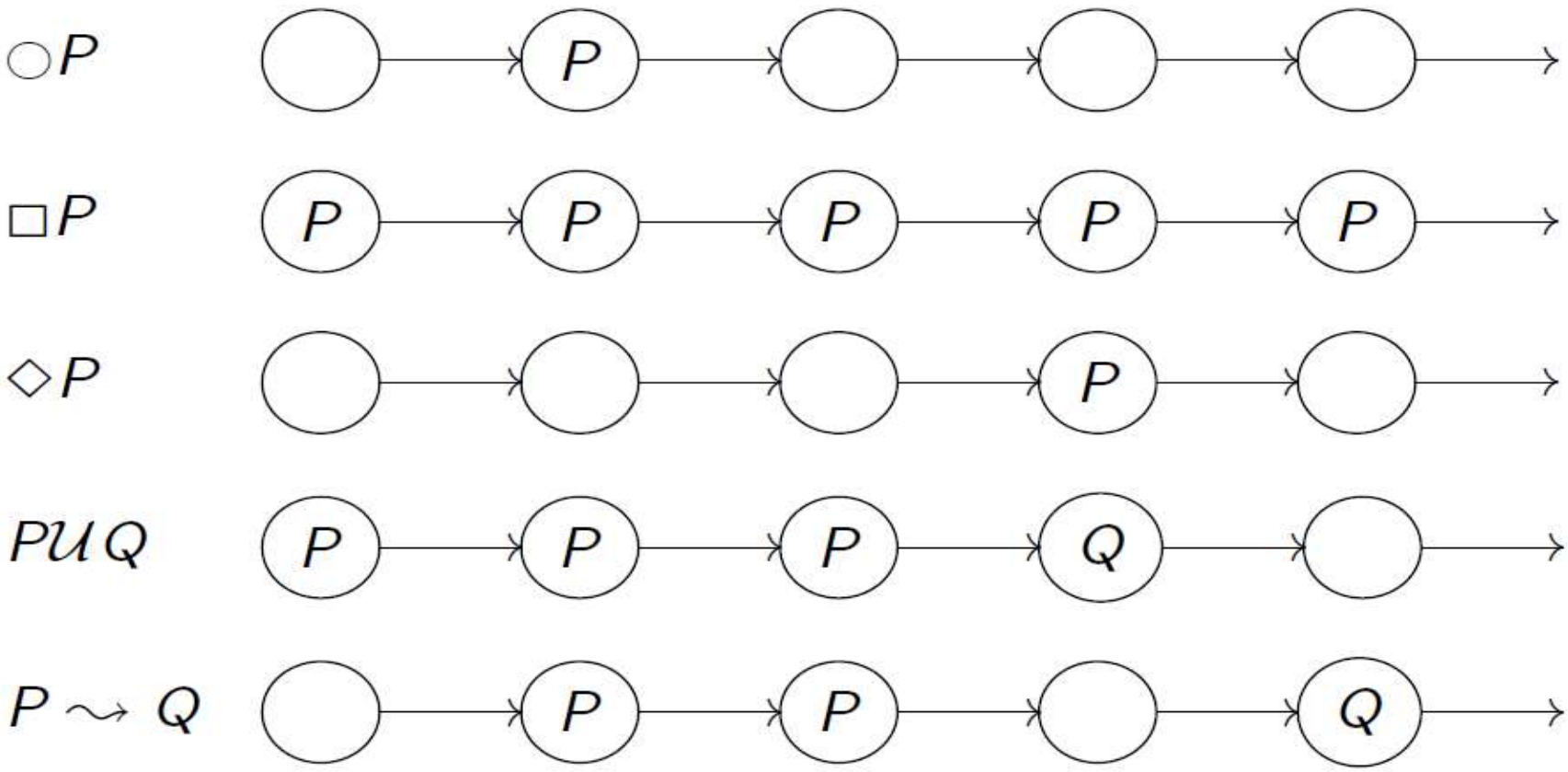
formule	nom	interprétation
$\bigcirc P$	next	P est vrai à l'instant suivant
$\Box P$	always	P est toujours vrai
		i.e. à tout instant à partir de l'instant courant
$\Diamond P$	eventually	P finit par être vrai (dans le futur)
$P \mathcal{U} Q$	until	Q finit par être vrai, et en attendant P reste vrai
$P \rightsquigarrow Q$	leadsto	quand P est vrai, alors Q l'est plus tard

Next noté aussi X, Future F, Always G (pour Globally) et Until $\mathcal{U}$

Langage de la logique temporelle

Logique temporelle : 3 Composants essentielles

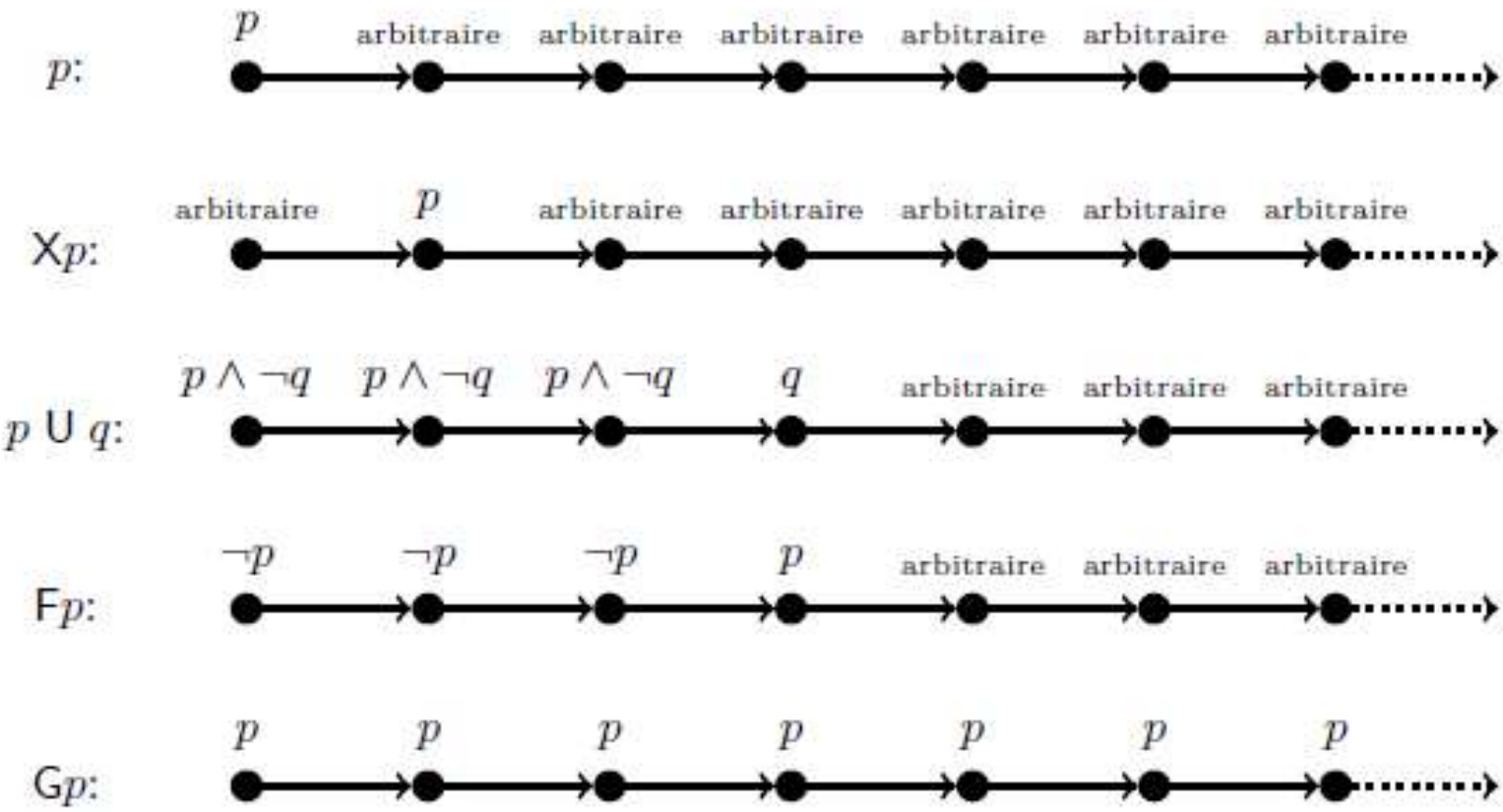
- Connecteurs temporels : Intuition sémantique



Langage de la logique temporelle

Logique temporelle : 3 Composants essentielles

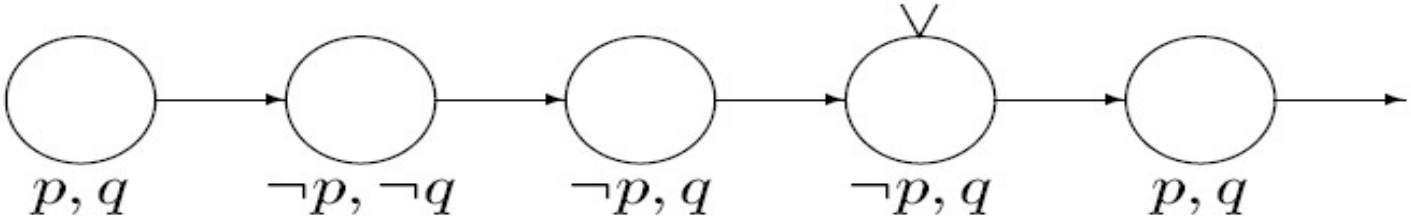
- Connecteurs temporels : Intuition sémantique



Langage de la logique temporelle

Logique temporelle : 3 Composantes essentielles

- Connecteurs temporels : Exemple



  - Dans le 4ème état : on peut observer que les propositions

    $\neg p \wedge q$ et $\bigcirc(p \wedge q)$ sont vraies

    $\Box p$ est fausse

- Opérateurs minimaux : $\{\bigcirc, \mathcal{U}\}$ forme un ensemble minimal d'opérateurs
  - $\Diamond p \equiv \text{true}\ \mathcal{U}\ p$
  - $\Box p \equiv \neg\Diamond\neg p$
  - $p \rightsquigarrow q \equiv \Box(p \Rightarrow \Diamond q)$

Langage de la logique temporelle

Logique temporelle : 3 Composantes essentielles

- Plus d'exemples :
  - $p \wedge \bigcirc p$: énonce p est vraie dans l'état courant et sera vraie dans l'état suivant
  - $\bigcirc\bigcirc p$: p vraie dans l'état suivant de l'état suivant
  - $\Box(p \Rightarrow \bigcirc\neg p)$: signifie que toujours quand p est vraie dans un état s, elle sera fausse dans l'état suivant de s
  - $\Box(p \Rightarrow \bigcirc q)$: signifie que toujours quand p est vraie dans s, q sera vraie dans l'état suivant de s
  - $\Box(p \Rightarrow \bigcirc(\neg q\ \mathcal{U}\ r))$: signifie que toujours quand p est vraie dans un état s, alors q reste fausse à partir de l'état suivant de s jusqu'à l'état où r est vraie

Langage de la logique temporelle

Logique temporelle : 3 Composantes essentielles

- **Opérateurs complémentaires :**

Opérateur	$\mathcal{W}$: Weak-Until ou le Waiting-for	$\mathcal{R}$: Release
Définition	$p \mathcal{W} q \equiv \Box p \vee p \mathcal{U} q$	$p \mathcal{R} q \equiv \Box q \vee q \mathcal{U} (p \wedge q)$
Sémantique	On exprime que p soit vrai jusqu'à ce que q le soit, mais sans exiger que q finisse par être vraie, et si q n'a jamais lieu, alors p restera indéfiniment vrai	informally means that q is true until p becomes true, or q is true forever.

Langage de la logique temporelle

Logique temporelle : 3 Composantes essentielles

Equivalence des formules de LT

duality law	*idempotency law*
$\neg \bigcirc \varphi \equiv \bigcirc \neg \varphi$	$\Diamond\Diamond\varphi \equiv \Diamond\varphi$
$\neg\Diamond\varphi \equiv \Box\neg\varphi$	$\Box\Box\varphi \equiv \Box\varphi$
$\neg\Box\varphi \equiv \Diamond\neg\varphi$	$\varphi \mathbf{U} (\varphi \mathbf{U} \psi) \equiv \varphi \mathbf{U} \psi$
	$(\varphi \mathbf{U} \psi) \mathbf{U} \psi \equiv \varphi \mathbf{U} \psi$
absorption law	*expansion law*
$\Diamond\Box\Diamond\varphi \equiv \Box\Diamond\varphi$	$\varphi \mathbf{U} \psi \equiv \psi \vee (\varphi \wedge \bigcirc(\varphi \mathbf{U} \psi))$
$\Box\Diamond\Box\varphi \equiv \Diamond\Box\varphi$	$\Diamond\psi \equiv \psi \vee \bigcirc\Diamond\psi$
	$\Box\psi \equiv \psi \wedge \bigcirc\Box\psi$

distributive law

$$\bigcirc(\varphi \mathbf{U} \psi) \equiv (\bigcirc\varphi) \mathbf{U} (\bigcirc\psi)$$

$$\Diamond(\varphi \vee \psi) \equiv \Diamond\varphi \vee \Diamond\psi$$

$$\Box(\varphi \wedge \psi) \equiv \Box\varphi \wedge \Box\psi$$

Langage de la logique temporelle

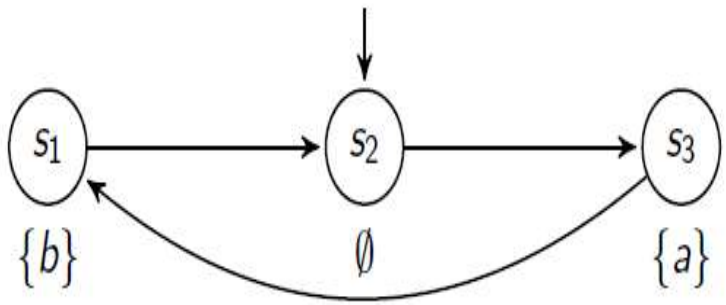
➔ Logique temporelle : 3 Composantes essentielles

- **Mais attention ……**

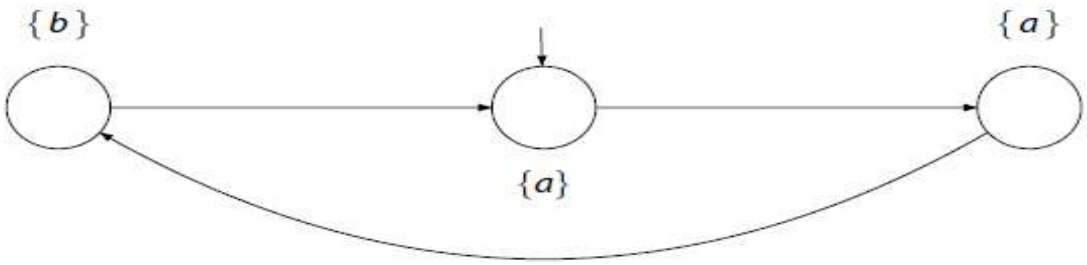
$$\Diamond(\phi \mathsf{U} \psi) \not\equiv (\Diamond\phi)\mathsf{U}(\Diamond\psi)$$

$$\Diamond(\phi \wedge \psi) \not\equiv \Diamond\phi \wedge \Diamond\psi$$

$$\Box(\phi \vee \psi) \not\equiv \Box\phi \vee \Box\psi$$



$TS \models \Diamond a \wedge \Diamond b$ mais $TS \not\models \Diamond(a \wedge b)$



$TS \not\models \Diamond(a \wedge b)$ and $TS \models (\Diamond a) \wedge (\Diamond b)$

$TS \not\models (\Box a) \vee (\Box b)$ and $TS \models \Box(a \vee b)$

Langage de la logique temporelle

➔ Logique temporelle : Fragment linéaire

Jusqu'à maintenant, on a considéré une seule exécution du système, les connecteurs permettent d'exprimer des propriétés le long de cette exécution. C'est une vision linaire du temps : le futur est fixé

☞ On parle de Logique Temporelle Linéaire (LTL)

Langage de la logique temporelle

➔ **Logique temporelle : Fragment branchant ou arborescent**

Le temps peut être aussi vu comme une structure branchante : à partir d'un même état, plusieurs futurs sont possibles (selon l'action qui sera effectuée).

☞ On parle de Logique arborescente (CTL)

Langage de la logique temporelle

Logique CTL : Computation Tree Logic

- La logique initiale de E. Clarke et E. Emerson

Professor Emeritus in the Dept. of Computer Science at Carnegie Mellon University,
ACM Turing Award in 2007

Regents Chair and Professor of Computer Science, University of Texas in Austin,
ACM Turing Award in 2007

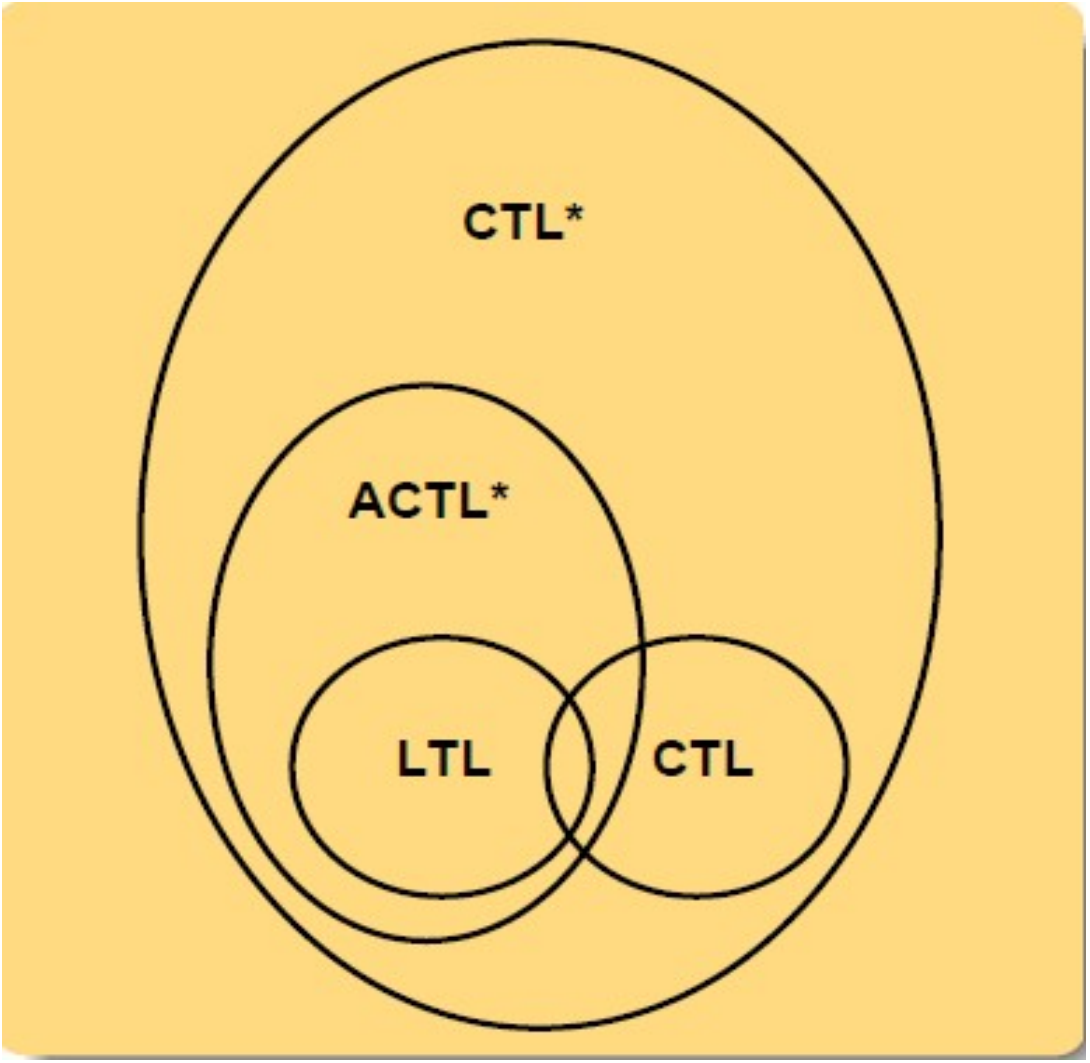
Langage de la logique temporelle

➔ Logique CTL : Computation Tree Logic

- **Logique temporelle arborescente (plusieurs futures sont possibles)**
- **Considère le système de transition non étiquetées plutôt que les traces du système.**
- **CTL est une restriction de la logique CTL* :** $LTL + CTL \subset CTL^*$
  - CTL * est une logique branchante très expressive
  - CTL* combine les opérateurs temporels linéaires et arborescents
    - ✓ Opérateurs arborescents : Quantificateurs de chemins (utilisés librement)
    - ✓ Ce qui permet d'exprimer des propriétés sur les arbres d'exécution

Langage de la logique temporelle

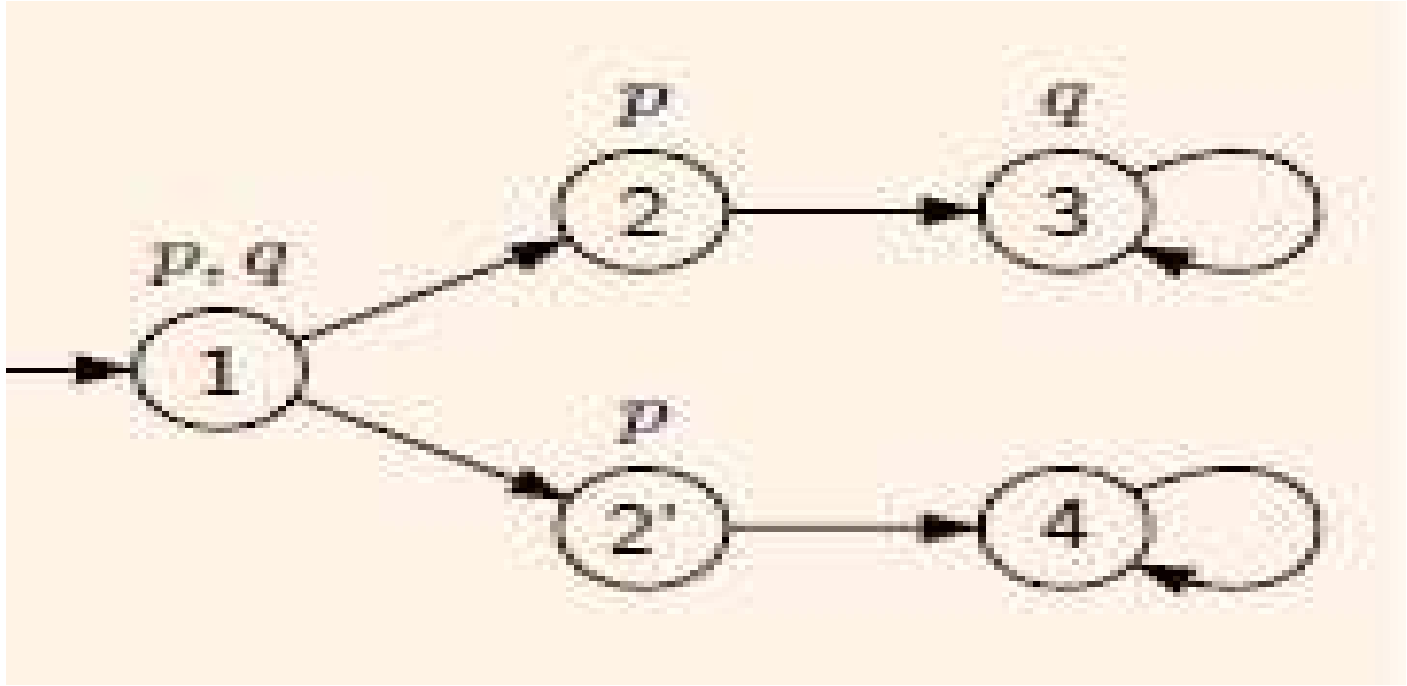
→ **Logique CTL : Computation Tree Logic**



Langage de la logique temporelle

Logique CTL : Computation Tree Logic

- Les chemins (ou comportements sont donc des parcours ou des branches particuliers dans l'arborescence des comportements



Langage de la logique temporelle

➔ Logique CTL : Computation Tree Logic

- **Opérateurs de chemins**
  - $A\varphi$: Signifie que toutes les exécutions, partant de l'état courant, satisfont la propriété φ
  - $E\varphi$: Signifie qu'il existe (au moins) une exécution, partant de l'état courant, qui satisfait la propriété φ
- **Ne pas confondre : $A\varphi$ et $\Box\varphi$ Opérateurs**
  - $A\varphi$: toutes les exécutions possibles maintenant satisfont φ
  - $\Box\varphi$: à tout instant (ou état) de l'exécution considérée φ est vérifiée
- **Liens entre quantificateurs de chemins**

$$\neg E\varphi \equiv A\neg\varphi$$

$$\neg A\varphi \equiv E\neg\varphi$$

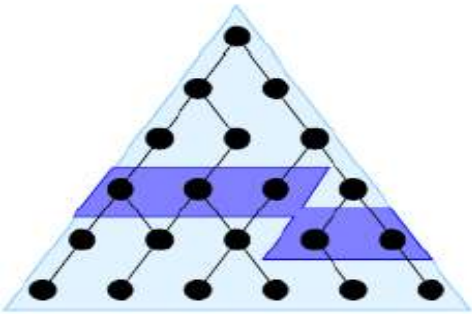
Langage de la logique temporelle

➔ Logique CTL : Computation Tree Logic

- **Un quantificateur de chemin peut être associé à une formule de chemin bâtie sur $\bigcirc, \Diamond, \Box, \mathcal{U}$ (avec s l'état courant)**
- **Opérateurs temporels linéaires $\bigcirc, \Diamond, \Box, \mathcal{U}$ doivent être immédiatement précédés d'opérateurs de chemins A, E**
- **Exemple**
  - EF(AGp) dans CTL
  - E(Gp ∧ Xq) pas dans CTL, AFGp pas dans CTL

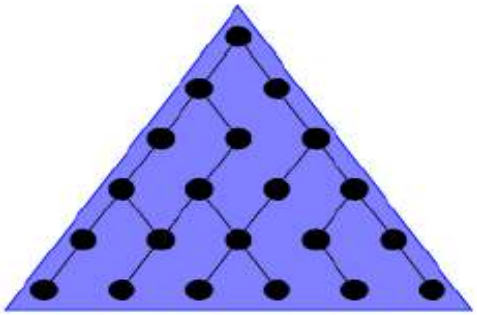
Langage de la logique temporelle

finally p



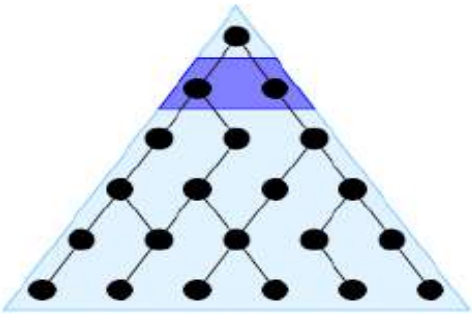
AF p

globally p



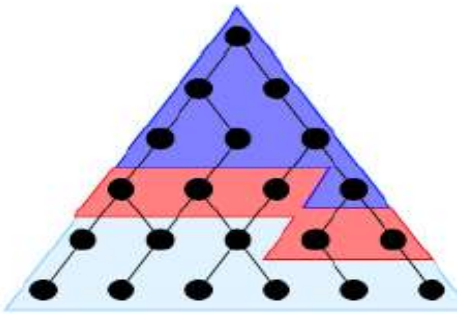
AG p

next p

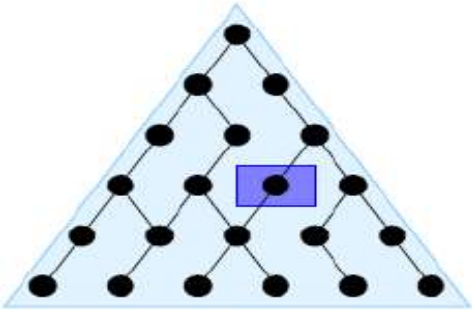


AX p

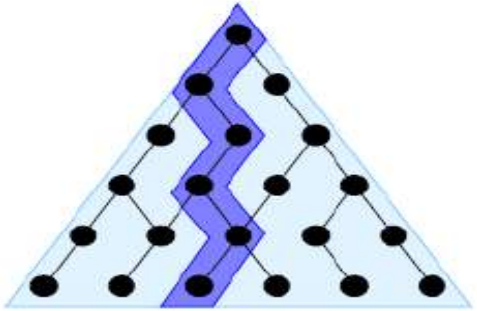
p until q



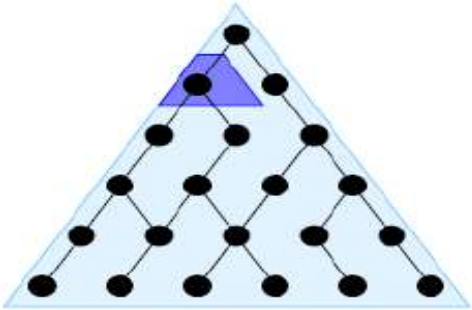
A [p U q]



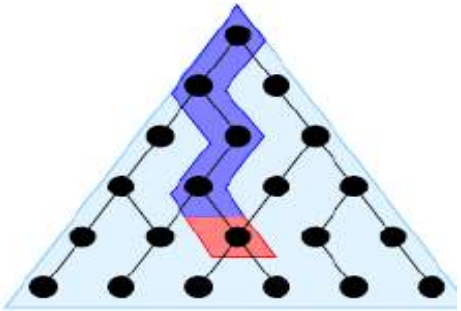
EF p



EG p



EX p



E [p U q]

source Alessandro Artale, http://www.inf.unibz.it/~artale/

Langage de la logique temporelle

➔ Logique CTL : Computation Tree Logic

Formule	Sémantique
A F p	Tous les chemins partant de s finissent par atteindre p
A G p	p est toujours vrai en partant de s (aussi vrai dans s)

E F p	∃ un chemin partant de s qui atteint p
E G p	∃ un chemin partant de s tel que p est vraie le long du chemin

Syntaxe de la logique temporelle

➔ LTL v.s. CTL

- Certaines formules de LTL ne peuvent pas être exprimées dans CTL
  - $FG\ p$
  - $F(p \wedge Xp)$
- Certaines formules de CTL ne peuvent pas être exprimées dans LTL
  - $AFAG\ p$
  - AGEF p

☞ **Ne peuvent pas être exprimées = il n'y a pas de formule équivalente**

Syntaxe de la logique temporelle

→ **Comparaison (LTL, CTL, CLT*)**

LTL, CTL, CTL* ?

	nature du temps	équité
LTL	linéaire	oui
CTL	arborescent	non
CTL*	linéaire et arborescent	oui

Syntaxe de la logique temporelle

Syntaxe de la logique temporelle

➔ Logique LTL : sur un chemin

- **Grammaire**

$$\varphi ::= p \in \mathrm{AP} \mid \neg\varphi \mid \varphi \vee \varphi \mid \varphi \wedge \varphi \mid \mathbf{X}\varphi \mid \mathbf{F}\varphi \mid \mathbf{G}\varphi \mid \varphi\mathbf{U}\varphi$$

AP : ensemble des propositions atomiques

{¬, ∧, ∨} : connecteurs logiques

{X, F, G, $\mathcal{U}$} : connecteurs temporels

- **Domaine d'interprétation :** ensemble de chemins σ

  - $\boldsymbol{\sigma} = s_0 s_1 s_2 \cdots s_n \cdots$: séquence infinie d'états s de Q
  - $\boldsymbol{\sigma}^i$: chemin extrait de $\boldsymbol{\sigma}$ à partir de la i-ème position, $\boldsymbol{\sigma}^2 = s_2 s_3 \cdots s_n \cdots$
  - $\boldsymbol{\sigma}(\mathbf{k})$: désigne le k-ième état de σ
  - Fonction $\boldsymbol{l} : \boldsymbol{Q} \rightarrow \mathbf{2}^{\boldsymbol{AP}}$ indique quelles propriétés $p \in AP$ sont vraies dans un s de Q

Syntaxe de la logique temporelle

➔ Logique LTL : sur une structure de Kriple

- **Modèle de Kriple :** $\mathcal{M} = (Q,\ \rightarrow,\ AP,\ l, s_0)$

  Q : ensemble des états

  $\rightarrow \subseteq \boldsymbol{Q} \times \boldsymbol{Q}$ **:** relation de transition

  AP : ensemble des propositions atomiques

  $\boldsymbol{l} : \boldsymbol{Q} \rightarrow \boldsymbol{2^{AP}}$ **:** fonction d'étiquetage des états (propriétés vraies)

  $\boldsymbol{s_0} \in \boldsymbol{Q}$ **:** l'état initial

- **Domaine d'interprétation :** un couple $(\mathcal{M}, s)$

- **Grammaire**

  - Formules LTL sur une structure de Kriple de la forme $A\varphi$ où φ est une formule LTL de chemin

$$\varphi_s ::= \mathbf{A}\varphi_p$$
$$\varphi_p ::= p \in \mathrm{AP}$$
$$\mid \neg\varphi_p \mid \varphi_p \vee \varphi_p \mid \varphi_p \wedge \varphi_p \mid \mathbf{X}\varphi_p \mid \mathbf{F}\varphi_p \mid \mathbf{G}\varphi_p \mid \varphi_p \mathbf{U} \varphi_p$$

Syntaxe de la logique temporelle

➔ Logique CTL* :

- **Formules** : Deux types
  - Formules d'état (φ_s), interprétées sur les états de la structure de Kriple
  - Formules de chemin (φ_p), interprétées sur les chemins de la structure de Kriple
- **Grammaire**

$$\varphi_s ::= p \in \mathsf{AP} \mid \neg\varphi_s \mid \varphi_s \vee \varphi_s \mid \varphi_s \wedge \varphi_s \mid \mathbf{A}\varphi_p \mid \mathbf{E}\varphi_p$$
$$\varphi_p ::= \varphi_s \mid \neg\varphi_p \mid \varphi_p \vee \varphi_p \mid \varphi_p \wedge \varphi_p \mid \mathbf{X}\varphi_p \mid \mathbf{F}\varphi_p \mid \mathbf{G}\varphi_p \mid \varphi_p \mathbf{U} \varphi_p$$

- **Domaine d'interprétation :** un couple ($\mathcal{M}$, s)

Syntaxe de la logique temporelle

➔ Logique CTL :

- **CTL restriction branchante de CTL***
  - A, E libres
  - X, F,G,$\mathcal{U}$ doivent être précédés directement de A, E
- **Formule CTL s'obtient à partir de**
  - de **{¬, ∧, ∨}**
  - et des opérateurs
    - ✓ AX et EX
    - ✓AF et EF
    - ✓AG et EG
    - ✓A$\mathcal{U}$ et E$\mathcal{U}$

Sémantique de la logique temporelle

Sémantique de la logique temporelle

➔ Relation de satisfaction / satisfaisabilité : $\models$

- **Modèle d'un système de transitions :** $\mathcal{M}$
- **Formule temporelle** φ
- **Questions :**
  - $\mathcal{M} \models \varphi$? : $\mathcal{M}$ valide – t-il (satisfait-il) φ ? φ est – elle vraie dans $\mathcal{M}$?
  - $\sigma \in \mathcal{M}$, $\sigma \models \varphi$? : σ valide – t-il (satisfait-il) φ ? φ est – elle vraie dans σ ?
- **Notation**
  - $\mathcal{M}, \sigma, i \models \varphi$: signifie au temps i (instant ou état) de l'exécution (calcul ou chemin) σ de $\mathcal{M}$, la formule φ est vraie.
  - On peut omettre $\mathcal{M}$ (le même système étudié) : $\sigma, i \models \varphi$

Sémantique de la logique temporelle

Formule LTL :

- **Satisfaisabilité sur un chemin σ**

$\sigma \models p$ iff $p \in l(\sigma(0))$

$\sigma \models \neg\varphi$ iff $\sigma \not\models \varphi$

$\sigma \models \varphi_1 \vee \varphi_2$ iff $\sigma \models \varphi_1$ ou $\sigma \models \varphi_2$

$\sigma \models \varphi_1 \wedge \varphi_2$ iff $\sigma \models \varphi_1$ et $\sigma \models \varphi_2$

$\sigma \models \mathbf{X}\varphi$ iff $\sigma^1 \models \varphi$

$\sigma \models \mathbf{F}\varphi$ iff il existe $k \geq 0$ tel que $\sigma^k \models \varphi$

$\sigma \models \mathbf{G}\varphi$ iff pour tout $k \geq 0$ on a $\sigma^k \models \varphi$

$\sigma \models \varphi_1 \mathbf{U} \varphi_2$ iff il existe $k \geq 0$ tel que $\sigma^k \models \varphi_2$ et pour tout $0 \leq j < k$ $\sigma^j \models \varphi_1$

Sémantique de la logique temporelle

Formule LTL :

- **Satisfaisabilité sur un couple ($\mathcal{M}$, s)**

. $\mathcal{M}, s \models_s \mathbf{A}\varphi_p$ ssi tous les chemins σ partant de s vérifient $\sigma \models \varphi_p$

. $\sigma \models \varphi_p$ défini comme avant

$\mathcal{M} \models \varphi$ ssi $\mathcal{M}, s_0 \models_s \varphi$

OU

$\mathcal{M} \models \varphi$ ssi, pour toute exécution $\sigma \in \mathcal{M}$, on a

$\sigma, s_0 \models \varphi$

Sémantique de la logique temporelle

→ Formule CTL* :

formules d'état, relation $\models_s$

- $\mathcal{M}, s \models_s p \in \mathrm{AP}$ iff $p \in l(s)$
- $\mathcal{M}, s \models_s \mathbf{A}f$ ssi tous les chemins σ partant de s vérifient $\mathcal{M}, \sigma \models_p f$
- $\mathcal{M}, s \models_s \mathbf{E}f$ ssi il existe un chemins σ partant de s vérifiant $\mathcal{M}, \sigma \models_p f$

formules de chemin, relation $\models_p$

- $\mathcal{M}, \sigma \models_p f$ iff $\mathcal{M}, \sigma(0) \models_s f$ (f formule d'état)
- $\mathcal{M}, \sigma \models_p \mathbf{X}\varphi$ iff $\mathcal{M}, \sigma^1 \models_p \varphi$
- $\mathcal{M}, \sigma \models_p \mathbf{F}\varphi$ iff il existe $k \geq 0$ tel que $\mathcal{M}, \sigma^k \models_p \varphi$

Sémantique de la logique temporelle

→ **Formule CTL* :**

$\mathcal{M} \models \varphi$ ssi $\mathcal{M}, s_0 \models_s \varphi$

OU

$\mathcal{M} \models \varphi$ ssi, pour toute exécution $\sigma \in \mathcal{M}$, on a

$\sigma, s_0 \models \varphi$

Sémantique de la logique temporelle

→ Formule CTL* :

formules d'état, relation $\models_s$

- $\mathcal{M}, s \models_s p \in \mathrm{AP}$ iff $p \in l(s)$
- $\mathcal{M}, s \models_s \mathbf{A}f$ ssi tous les chemins σ partant de s vérifient $\mathcal{M}, \sigma \models_p f$
- $\mathcal{M}, s \models_s \mathbf{E}f$ ssi il existe un chemins σ partant de s vérifiant $\mathcal{M}, \sigma \models_p f$

formules de chemin, relation $\models_p$

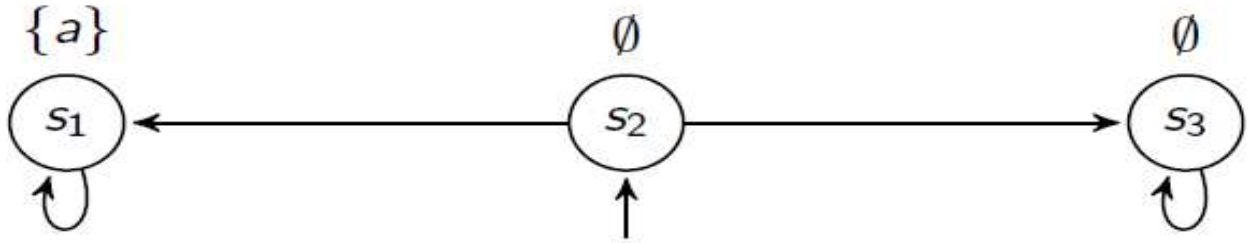
- $\mathcal{M}, \sigma \models_p f$ iff $\mathcal{M}, \sigma(0) \models_s f$ (f formule d'état)
- $\mathcal{M}, \sigma \models_p \mathbf{X}\varphi$ iff $\mathcal{M}, \sigma^1 \models_p \varphi$
- $\mathcal{M}, \sigma \models_p \mathbf{F}\varphi$ iff il existe $k \geq 0$ tel que $\mathcal{M}, \sigma^k \models_p \varphi$

Sémantique de la logique temporelle

Exemples

Négation

- $\sigma \models \neg P \equiv \sigma \not\models P$ ($\not\models$: ne valide pas ou ne satisfait pas)
- $\mathcal{M} \not\models \varphi$: Il existe au moins une exécution (chemin) qui invalide φ mais pas toutes les exécutions le font.
- En LTL, on peut avoir : $\mathcal{M} \not\models \varphi \wedge \mathcal{M} \not\models \neg\varphi$
  - $\Diamond a$? $\neg\Diamond a$?

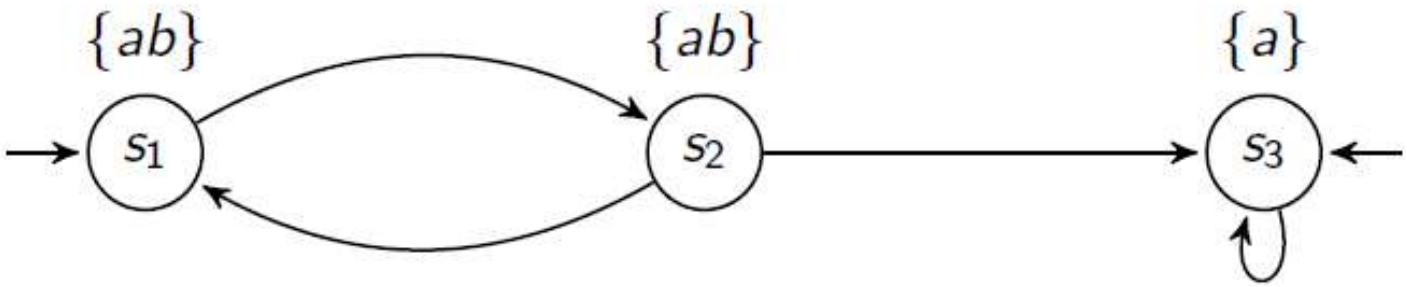


Un système de transitions TS pour lequel $TS \not\models \Diamond a$ et $TS \not\models \neg \Diamond a$

Sémantique de la logique temporelle

Exemples

- **Exemple**



$$TS \models \Box a$$

$$TS \not\models X(a \wedge b)$$

$$TS \models \Box(\neg b \rightarrow \Box(a \wedge \neg b))$$

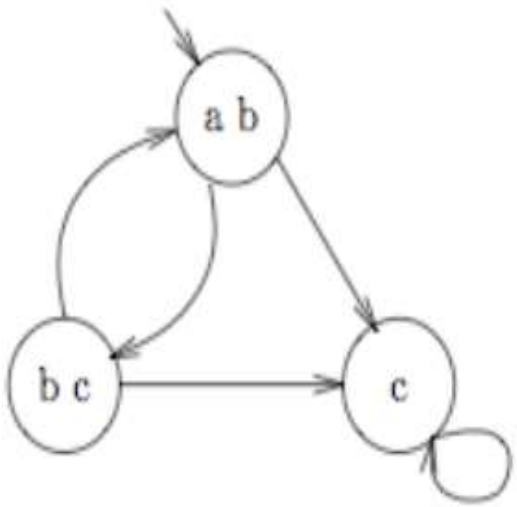
Sémantique de la logique temporelle

Exemples

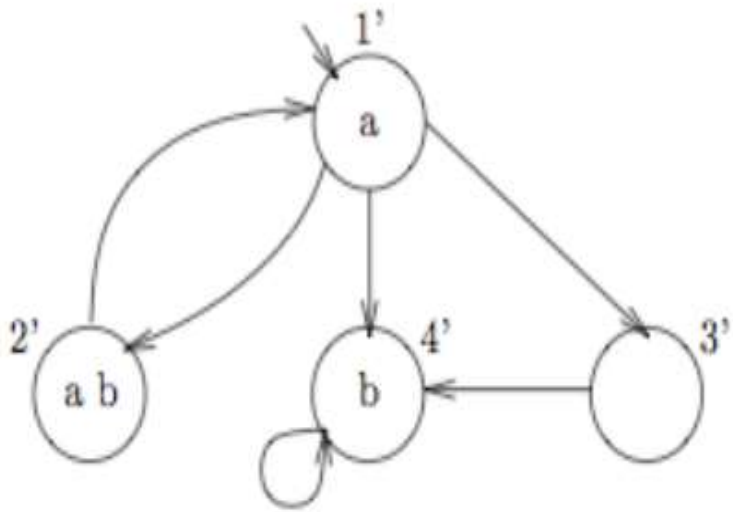
Exemple 1 :

Indiquez quelle est la valeur de vérité pour les deux formules LTL suivantes par rapport aux structures de Kripke ci-dessous.

1. $\Box(b \Rightarrow \Diamond b)$
2. $a \mathcal{U} b$



M1



M2

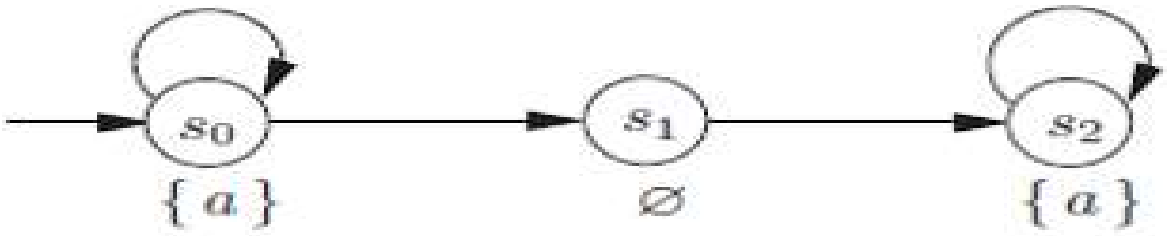
Sémantique de la logique temporelle

Exemples

Exemple 2 :

Soit le système de transitions ST ci-dessous.

Soit la formule logique φ = a. Montrer que les deux formules **AFAG**φ et **FG**φ ne sont pas équivalentes.



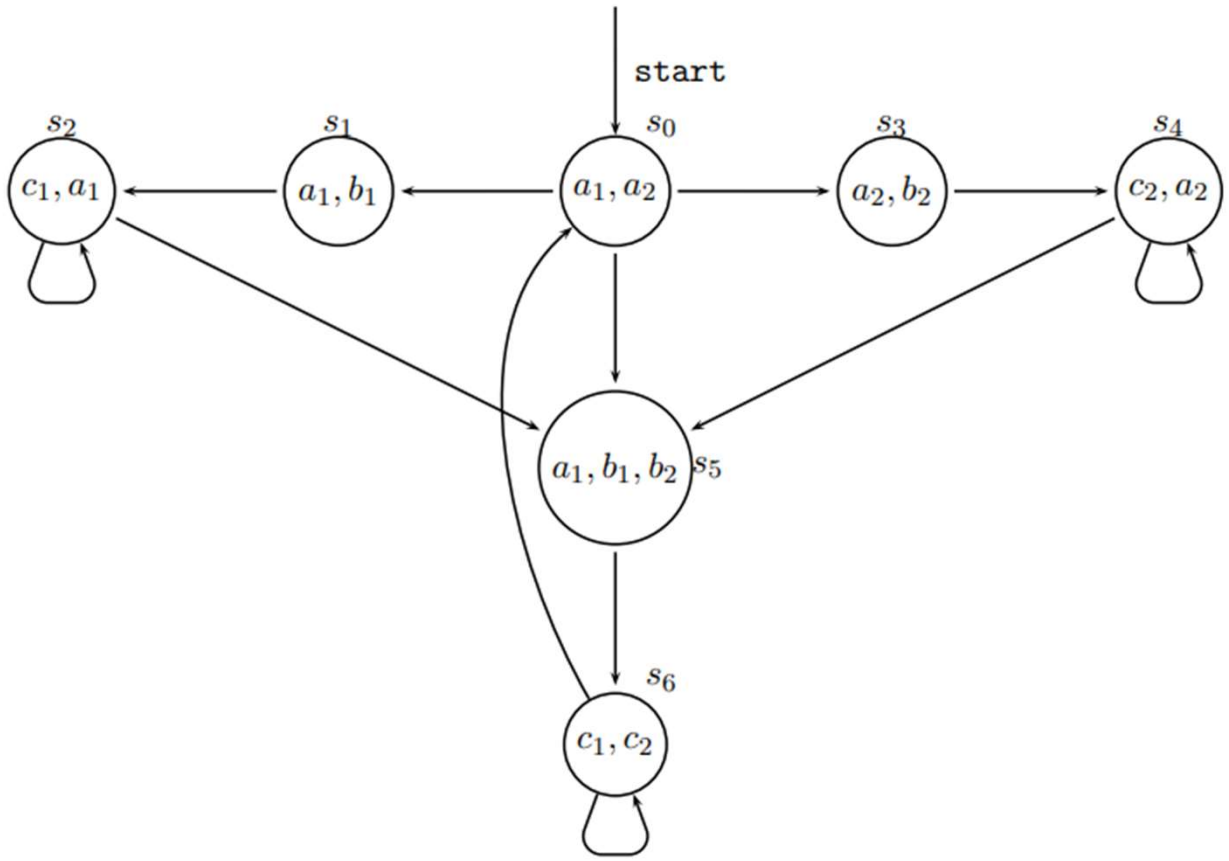
La formule LTL est vraie, mais la formule CTL est fausse. La formule LTL est vraie pour cet automate, car toute trace infinie contient toujours un suffixe ne comportant que des a. Une trace infinie boucle soit sur l'état s_0 ou sur l'état s_2. Dans les deux cas, on a une suite infinie de a dans le suffixe de la trace. La formule CTL est fausse, car une trace qui boucle sur s_0 offre toujours une branche menant à s_1 où a n'est pas satisfaite.

Sémantique de la logique temporelle

Exemples

Exemple 3 :

You are given the following Kripke model M :



Sémantique de la logique temporelle

Exemples

Exemple 3 (Cont):

Extract from the above graphical representation of $\mathcal{M}$ its formal definition.
Furthermore, for each of the following **LTL** formulas φ:

1. $((a_1 \wedge \neg b_2) \vee b_1) \rightarrow \bigcirc b_2 \vee \bigcirc \bigcirc (c_1 \vee c_2)$
2. $\Box(\neg c_2 \vee \bigcirc a_1)$
3. $\Box(\bigcirc b_1 \rightarrow \Diamond(c_1 \wedge \neg b_1))$
4. $\Box\Diamond b_2 \rightarrow \Box\Diamond(a_1 \wedge a_2)$
5. $(a_1 \vee a_2)\,\mathcal{U}\,(c_1 \vee c_2)$

reply to the following questions:

1. Find a path from the initial state which satisfies φ.
2. Check whether $\mathcal{M} \models \varphi$, and in case the answer is negative exhibit a path that does not satisfy the formula.

Algorithmes de Model-Checking

Reste à étudier les algorithmes de Model-Checking pour LTL et CTL